```
// valid palindrome  --   using  recursion
// hints: two pointers, check characters from start and end moving towards center
function isPalindrome(str) {

    function helper(str, l, r) {
        // Base case: crossed pointers !’ palindrome
        if (l >= r) return true;

        if (str[l] !== str[r]) return false;

        return helper(str, l + 1, r - 1);
    }

    return helper(str, 0, str.length - 1);
}

console.log(isPalindrome('racecar'));  // true
console.log(isPalindrome('hello'));    // false

// ---------------------------------------------------------------------

//longest common prefix  -- optimized( from the array of strings, find the longest common prefi
x string amongst them)
// hints: compare characters of first string with all others at same index
// time complexity -- O(nxm)  n = number of strings, m = length of first string

/**
 * @param {string[]} strs
 * @return {string}
 */
var longestCommonPrefix = function(strs) {
    let first = strs[0]
    let prefix = "";
    for (let i =0; i < first.length; i++) {
        let char = first[i];

        for (let j = 1; j < strs.length; j++) {
          // check if index i is out of bounds for strs[j] or characters don't match
            if (i >= strs[j].length || strs[j][i] !== char) {
                return prefix;
            }

        }
            prefix+=char; // add after second loop runs
    }

    return prefix;
};

//-------------------------------------------------------------------------------------
// group anagrams  -- optimized
// hints: use character count as key in map to group anagrams together
// time complexity -- O(n * k)  n = number of strings, k = length of longest string

/**
 * @param {string[]} strs
 * @return {string[][]}
 */
var groupAnagrams = function(strs) {
    let map = new Map();

    for (s of strs) {
        let count = new Array(26).fill(0);
        for (let i = 0; i< s.length; i++) {
            count[s.charCodeAt(i)- 97]++;
        }

        let key = count.join('#')

        if(!map.has(key)) map.set(key, []);
        map.get(key).push(s)
    }

    return Array.from(map.values())
};
```

```
// ------------------------------------------------------------------------
// valid anagram  -- optimized
// hint: use frequency array to count characters
// time complexity -- O(n) -- n is length of the anagram


function isAnagram(s, t) {
  // quick length check
  if (s.length !== t.length) return false;

  // frequency array for 26 lowercase letters
  const counts = new Array(26);
  for (let i = 0; i < 26; i++) counts[i] = 0;

  const base = 'a'.charCodeAt(0);

  // count chars in s (increment) and t (decrement)
  for (let i = 0; i < s.length; i++) {
    counts[s.charCodeAt(i) - base] += 1;
    counts[t.charCodeAt(i) - base] -= 1;
  }

  // if all zeros, they're anagrams
  for (let i = 0; i < 26; i++) {
    if (counts[i] !== 0) return false;
  }
  return true;
}

// brute force

var isAnagram = function(s, t) {

        let count1 = new Array(26).fill(0);
      for (let i =0; i< s.length; i++) {
        count1[s.charCodeAt(i) - 97]++;
      }

      let key1 = count1.join('#');

        let count2 = new Array(26).fill(0);
       for (let i =0; i< t.length; i++) {
        count2[t.charCodeAt(i) - 97]++;
      }

      let key2 = count2.join('#');

      return key1 === key2

};

//------------------------------------------------------------------------
// longest palindromic substring  -- optimized
// time complexity -- O(n^2)  n = length of the string
// space complexity -- O(1)
// hint : expand around center technique

/**
 * @param {string} s
 * @return {string}
 */
var longestPalindrome = function(s) {
    if (s.length < 2) return s;

    let start = 0;
    let maxLen = 1;

    function expand(left, right) {
        while(left >=0 && right < s.length && s[left] === s[right]) {
            let currLen = right-left+1;
            if (currLen > maxLen) {
                maxLen = currLen;
                start = left;
            }
            left--;
            right++
```

```
        }
    }


    for (let i = 0; i < s.length; i++) {
        expand(i,i);
        // even length
        expand(i, i+1)
    }

    let result = "";
    for (let i=start; i< start + maxLen;i++ ) {
        result+=s[i]
    }

    return result;
};
```